Field Hours

CSCE 633 Machine Learning

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 45 minutes

Supervised induction and linear models

Induction here means inferring a function from examples. A training set is pairs (x, y). A linear model predicts a weighted sum of the coordinates of x, plus a bias. Least squares chooses the weights that minimize the sum of squared differences between prediction and y. Squared error is not sacred. It is convenient, it punishes large misses, and it has a closed form when the columns of the design matrix are linearly independent.

Stack the examples as rows of a matrix X, with a column of ones for the bias. The normal equations are X transpose X times the weight vector equals X transpose y. You solve a square system whose side is the number of features, not the number of examples. If two features are copies of each other, X transpose X is singular and there is no unique weight vector. Drop a feature or add a ridge term.

The fit on the training points is not the result. The result is the prediction on a new x drawn the same way. A line that threads every training point in a high-dimensional space can be an interpolating accident. Keep a test set out of the solver. If you only have twenty points, say so next to the number.

Explanation-based learning is the other induction in this catalog. You already have a theory, and one example is enough to specialize it into a rule. Statistical least squares is what you do when you do not have that theory. Do not pretend a fitted line is an explanation. It is a function with a measured error.

Least squares. The design matrix already includes the column of ones.

Build X -> Form X'X -> Solve weights -> Test on new x

Three points, prediction a x + b

1. Points (1, 2), (2, 3), (3, 5). X transpose X is the matrix [[3, 6], [6, 14]]. X transpose y is [10, 23].

2. Equations: 3b + 6a = 10 and 6b + 14a = 23.

3. From the first, b = (10 - 6a) / 3. Substitute: 20 - 12a + 14a = 23, so 2a = 3 and a = 1.5. Then b = 1/3.

4. Predictions are about 1.83, 3.33, and 4.83. The residuals are small and not zero. That is a fit, not a law.

Problem 1. Fit y = a x through the origin to the same three points. What is a?

Answer. a = (1*2 + 2*3 + 3*5) / (1 + 4 + 9) = 23 / 14, about 1.64. No bias term, so the line is steeper than 1.5 and misses the intercept the data wanted.

Problem 2. Two features are identical on every row. Why do the normal equations fail, and what is one repair?

Answer. The columns of X are dependent, so X transpose X has determinant 0 and no unique

solution. Drop one feature, or add a positive number to the diagonal (ridge regression) so the matrix is invertible.

Lecture 2 · 45 minutes

Classification, bias, and variance

Logistic regression classifies by turning a weighted sum into a probability through the logistic function, one over one plus e to the minus z. Training minimizes the negative log likelihood, which is convex in the weights. There is no closed form. Gradient descent takes a step opposite the gradient. On a small problem you should compute one step by hand so the learning rate is a number you chose, not a ritual.

Bias is error from a hypothesis that cannot represent the pattern even with infinite data. A line cannot fit a clean parabola, and more points will not save it. Variance is error from a hypothesis that changes too much when the sample changes. A high-degree polynomial can pass through every training point and swing wildly between them. The test error is the sum of those two pressures plus noise you cannot remove.

Regularization adds a penalty on the size of the weights. Ridge adds the sum of squares of the weights. It shrinks every weight and usually lowers variance and raises bias. The penalty strength is chosen on a validation set. Lasso uses the sum of absolute weights and can set some weights to exactly zero, which is a form of feature selection. Neither penalty knows which features you care about for reasons outside the loss.

A model with lower training error is not automatically better. Plot both training and validation error against model capacity. When they move apart, you are fitting the sample. Stop, or penalize, or get more data. More data reduces variance. It does not fix a hypothesis class that cannot represent the pattern.

Capacity is the knob. Both ends lose, for different reasons.

High bias

Line on a curve

Train error stays up

More data will not fix it

High variance

A kink per point

Train error near zero

A new sample moves the fit

One gradient step on a single logistic point

1. One feature x = 1, label y = 1, weight w = 0, bias 0, so z = 0 and the predicted probability is 0.5.

2. The gradient of the log loss with respect to w, for one point, is (probability minus y) times x = (0.5 - 1) * 1 = -0.5.

3. A step with learning rate 0.2 moves w from 0 to 0.1, toward the positive label.

4. The new probability is logistic(0.1), a little above one half. One step is not training. It is a check that

the sign is right.

Problem 1. Training error falls and validation error rises as you add polynomial degree. Which side of the pair are you on, and what do you change?

Answer. High variance. Reduce degree, strengthen regularization, or add data. Do not add features.

Problem 2. A line's validation error stays high and matches its training error, on data that is clearly curved. What do you change?

Answer. The class is biased. Increase capacity, for example a quadratic term. More copies of the same points will not draw the curve.

Lecture 3 · 40 minutes

Clustering, exemplars, and analogy

Clustering is unsupervised. You have points and no labels, and you want groups. k-means picks k centers, assigns every point to the nearest center, and moves each center to the mean of its points. Repeat until the assignment stops changing. The objective, total squared distance to centers, never rises. It can stop in a bad local minimum, so run it more than once from different starts and keep the best objective. k is not estimated by the algorithm. You chose it.

An exemplar method stores points instead of fitting weights. The nearest neighbor classifier predicts the label of the closest stored point. k nearest neighbors vote among k of them. The method is analogy: this case resembles that case. It has almost no training time and a prediction time that grows with the sample unless you build an index. The distance is the model. Euclidean distance on raw pixels is a poor analogy. Euclidean distance on features you designed might be a good one.

A single stored prototype per class is the other extreme. Compute the mean of each class and classify by the nearer mean. That is a linear method in disguise for two classes, and it fails when a class is two blobs. k-means on each class separately is one way to admit that a class has more than one exemplar.

Discovery in this catalog means noticing a structure you did not label in advance. A cluster is a proposal, not a finding, until you can predict something new with it. If the groups do not correspond to any measurement you held out, you have partitioned a sample. Say that in the write-up.

Two unlabeled pictures of the same points. k is a decision.

k = 2

Two means

Lower detail

May merge a real split

k = 4

Four means

Fits small clumps

May split noise

Four points on a line

1. Points at 0, 1, 10, and 11. Start with centers at 0 and 10.

2. Assign 0 and 1 to the first center, 10 and 11 to the second.

3. New centers: 0.5 and 10.5. The assignment does not change. Stop.

4. A start with both centers at 0 and 1 can collapse. That is why you restart.

Problem 1. Nearest neighbor on a training set memorizes every training label. Why can test accuracy still be poor?

Answer. Prediction uses distance, not the training label of the test point. If the distance is wrong, the neighbor is the wrong analogy. Noise labels are also copied exactly. Accuracy on the training set is 100 percent by construction and is not evidence.

Problem 2. k-means objective after iteration 3 is 14, and after iteration 4 is 16. Where is the bug?

Answer. A correct k-means step never increases the sum of squared distances. The assignment step or the mean step is wrong, or the distance changed between iterations. Do not interpret 16 as a worse cluster you discovered. It is an implementation error.

Lecture 4 · 45 minutes

A network with one hidden layer

A feedforward network composes linear sums and a nonlinearity. Without the nonlinearity, a stack of linear maps is one linear map, and you have the first lecture again with worse notation. A hidden layer with a pointwise nonlinearity can approximate more functions as you add units. That existence claim does not say that gradient descent will find the approximation, or that you have the data to pin it down.

Backpropagation is the chain rule on a computation graph. You compute the output, compare it to the target, and send a derivative backward through each operation. Each weight's gradient is the local input to that weight times the derivative arriving from above. The arithmetic is ordinary. The bookkeeping is the part people skip, and it is the part that has to be right.

A worked linear network is enough to see the signs. Let the hidden value be w1 times x, the prediction be w2 times the hidden value, and the loss be one half the square of prediction minus target. The derivative of the loss with respect to the prediction is the residual. Multiply by the hidden value to get the gradient for w2. Multiply the residual by w2 and then by x to get the gradient for w1. A nonlinearity inserts its derivative in the middle and changes nothing about the shape of the argument.

Training still overfits. A network with enough units can memorize a finite sample. Early stopping, weight decay, and a validation set are the same bias-variance tools as in the linear lecture. A lower training loss is not the result you report.

Forward is left to right. The gradients travel back along the same pipes.

Input x -> Hidden h -> Output y -> Loss

One backprop step, no nonlinearity

1. x = 2, w1 = 0.5, so h = 1. w2 = 3, so the prediction is 3. Target is 1.

2. Loss is one half of (3 - 1) squared, which is 2. The residual is 2.

3. Gradient for w2 is residual times h = 2. Gradient for w1 is residual times w2 times x = 2 * 3 * 2 = 12.

4. A step of size 0.01 subtracts 0.02 from w2 and 0.12 from w1. The prediction moves toward 1.

Problem 1. In that example, what is the gradient for w1 if the target is 3 instead of 1?

Answer. Prediction equals target, residual is 0, and every gradient is 0. The step does nothing. That is correct, not a dead network.

Problem 2. You remove the nonlinearity and add a third linear layer. Can the training loss be lower than the best single linear map on the same features?

Answer. No. Three linear maps compose to one linear map. The minimum squared loss cannot beat the ordinary least-squares line on those features. Any gap is optimization, not representation.

Lecture 5 · 40 minutes

Evaluation and what a model does not show

Split the data before you look at it for decisions. Training fits the weights. Validation chooses the model. Test is touched once. If you touch the test set twice, the second number is not a test. Time series and medical records break a random split, because tomorrow's row is not independent of today's, and two rows from the same person are not two people. Split by time, or by person, or you will publish an echo.

Accuracy hides the base rate. A test that always says 'no disease' is 99 percent accurate when 1 percent of people have the disease, and it is useless. Precision is the fraction of positive predictions that are right. Recall is the fraction of real positives you caught. Report both, or report a confusion matrix. A single threshold is a decision about which error is more expensive, not a property of the model.

Calibration is whether a predicted 0.7 happens about 70 percent of the time. A classifier can rank well and still be overconfident. If you will threshold the probability or show it to a person, check calibration on the validation set. A pretty ranking curve does not answer that.

Discovery and analogy do not survive a bad split. A cluster found on the whole data set, then 'confirmed' on the same rows, is not a confirmation. An exemplar retrieved from the test patient's own history is leakage. Write the split in the first paragraph of the report, not the last.

Touch the layers in order. The test layer is not a place to choose a threshold.

Test, once

Validation, for choices

Training, for weights

A 100-person test, 10 of whom are positive

1. The model flags 10 people, and 5 of them are truly positive.

2. Precision is 5/10 = 0.5. Recall is 5/10 = 0.5.

3. Accuracy is 90 percent if the 5 false positives and 5 false negatives are the only mistakes: 90 correct out of 100.

4. The accuracy sounds like success. The recall says half the people you care about were missed.

Problem 1. A spam filter flags nothing. Spam is 5 percent of mail. What is the accuracy, the precision, and the recall?

Answer. Accuracy is 95 percent. Precision is undefined or irrelevant, because there are no positive predictions. Recall is 0. The accuracy is the base rate of non-spam.

Problem 2. You randomly split sensor readings taken every second. Why can the test error be optimistic?

Answer. Adjacent seconds are almost the same event. The test set contains near-copies of the training set. Split by day, or by device, so the test is a new period rather than the next second.

Lecture 6 · 180 minutes

Regularization

A model that fits the training set and fails on new data has memorized. Regularization adds a penalty so that large weights are expensive. Ridge regression minimizes the squared error plus lambda times the sum of squared weights. Lasso uses the sum of absolute weights and can set some weights to zero. The penalty is part of the objective. It is not a post-processing step you mention in a caption.

Lambda is chosen on validation data, never on the test set. If you try ten lambdas on the test set and keep the best, the test set became training data and the number you report is optimistic. Nested selection exists because of this. Use it, or hold out the test set until the model, the features, and lambda are frozen.

Weight decay in a neural net is ridge regression on the weights, usually excluding the biases. Dropout is a different regularizer: it randomly hides units during training so the network cannot depend on one path. At evaluation you use the full network and scale appropriately, or you use the inference rule your library documents. Mixing the training rule with the evaluation rule silently changes the model.

More data reduces variance. A penalty reduces variance by restricting the hypothesis class, and it can increase bias. The bias-variance lecture already said this. Regularization is how you act on it when you cannot collect more data this week.

The same fit, two penalties.

Ridge

Penalty on squared weights

Shrinks, rarely zeros

Closed form with the design matrix

Lasso

Penalty on absolute weights

Can zero a coefficient

No simple closed form in general

Three points and a penalty

1. You fit a line to three points and the unpenalized slope is 5 because one point is extreme.

2. Ridge with a large lambda pulls the slope toward 0. The training error rises. If the extreme point was noise, the validation error falls.

3. If the extreme point was the only signal, the same lambda raises validation error. The penalty does not know which case you are in. The validation set is how you tell.

Problem 1. You choose lambda by the lowest error on the test set, then report that error. What is wrong with the report?

Answer. The test error is no longer an unbiased estimate of future error. You used it to make a decision. Report the error on a set that was not used to pick lambda, or admit that the number is a training result.

Problem 2. Lasso sets a coefficient to zero. Does that mean the corresponding feature is irrelevant in the world?

Answer. No. It means that under this penalty and this sample, the objective preferred zero. A correlated feature can be zeroed while its partner carries the signal. Zero is a statement about the fit, not a discovery that the variable does not exist.

Lecture 7 · 180 minutes

Margins and kernels

A support vector machine for two classes finds a linear separator that maximizes the margin, the distance from the line to the nearest training point. The points that touch the margin are the support vectors. Moving a point that is not a support vector, as long as it stays on the correct side and outside the margin, does not move the separator. That is the geometric content of the method.

If the classes overlap, the soft-margin version allows violations and charges them in the objective. The charge is a hyperparameter, chosen on validation data, with the same discipline as lambda. A hard margin on overlapping data is infeasible. The solver is not broken. The constraint is.

A kernel is a dot product in some feature space. You never write the features down if the kernel can be computed from the original inputs. The polynomial kernel and the Gaussian kernel are the two you should be able to define. The Gaussian kernel can separate almost anything if its width is tiny, which means it can also memorize. Width is a hyperparameter.

The kernel trick does not say that a nonlinear problem became linear in the original coordinates. It became linear in a different space. Report the kernel you used. 'SVM' is not a specification.

A margin in the plane.

What moves the line

A point on the margin

A point on the wrong side

The penalty on violations

What does not

A point far on the correct side

A label you did not include

The filename of the dataset

Four points

1. Positive points at x = 0 and x = 1. Negative points at x = 3 and x = 4. In one dimension the hard-margin separator sits at the midpoint of the gap, x = 2.

2. The margin is the distance from 2 to the nearest point, which is 1. The support 'vectors' are the points at 1 and 3.

3. Move the point at 0 to -10. It is not a support vector. The separator stays at 2. Move the point at 1 to 1.5 and the separator moves, because that point defined the margin.

Problem 1. Two classes overlap in the original coordinates and you use a hard-margin linear SVM. What should the solver report?

Answer. Infeasible. There is no line with a positive margin that classifies every point correctly. Switch to a soft margin or a kernel, and say which. Do not add a tiny margin and call it hard.

Problem 2. A Gaussian kernel with a very small width classifies the training set perfectly and fails on a point placed next to a training point. What happened?

Answer. The kernel treated each training point as its own island. The model memorized locations. Widen the kernel using validation data, or you have a nearest-neighbor classifier with a fragile radius, not a margin method.

Lecture 8 · 180 minutes

Trees and ensembles

A decision tree splits the input on one feature at a time, chosen to reduce an impurity measure such as Gini or entropy. The tree can represent interactions that a linear model cannot, and it can also isolate single training points if you let it grow. A pure leaf on one training point is a memorization. Limit the depth, or require a minimum number of samples in a leaf, and choose that limit on validation data.

A single tree is unstable: a small change in the sample can change the root split. Bagging trains many trees on bootstrap samples and averages them. Random forests also restrict each split to a random subset of features, which makes the trees disagree more, which is what makes the average useful. Averaging models that all make the same mistake does nothing.

Boosting trains the next tree on the residuals or on reweighted points that the current ensemble gets wrong. Gradient boosting is the version you will meet in practice. It can overfit. The number of trees and the learning rate are hyperparameters. Early stopping on a validation set is the usual control.

Feature importance from a tree is a description of that fit, not a causal ranking. A feature used at the root reduced impurity in this sample. A correlated feature that was never chosen can be just as informative. Do not put an importance bar chart in a report without that sentence.

One tree, then a forest.

Root split on one feature

Leaves hold class counts

A second tree sees a bootstrap sample

The forest averages the votes

Four points, one split

1. Points: (0, red), (1, red), (2, blue), (3, blue), using a single numeric feature.

2. A split at 1.5 puts both reds on the left and both blues on the right. Impurity of each child is 0. The tree stops.

3. A split at 0.5 isolates one red and leaves the other red mixed with the blues. Impurity falls less. The first split is the one the algorithm keeps.

Problem 1. You grow a tree until every leaf is pure on the training set. What do you expect on a test point that falls into a leaf with one training sample?

Answer. The leaf predicts that sample's label, whether or not the region means anything. Training accuracy is 100 percent by construction. Test accuracy is whatever memorization is worth on this problem. Constrain the tree.

Problem 2. All 200 trees in a forest produce the same splits. Will the forest beat one tree?

Answer. No. The average of identical predictions is that prediction. Random feature subsets and bootstrap samples exist to make the errors differ. If they do not differ, you have one tree with extra steps.

Lecture 9 · 180 minutes

Mixtures

A mixture model says each point was produced by one of several sources, and you do not observe which. A Gaussian mixture uses a mean, a variance, and a weight for each source. Clustering from the earlier lecture is the special case where you assign each point to the nearest mean and the variances are fixed. The mixture also says how unsure that assignment is.

Expectation-maximization alternates two steps. The E-step computes the probability that each source produced each point, given the current parameters. The M-step updates the parameters from those probabilities as if they were fractional counts. Each full step does not decrease the likelihood. It can converge to a poor local maximum, so the start matters. Initializing means at random data points is the ordinary start, and you run more than once.

A variance that collapses to zero on a single point sends the likelihood to infinity. That is a known degeneracy of Gaussian mixtures, not a sign that you found a perfect cluster. Constrain the variance, or stop treating an infinite

likelihood as success.

The number of sources is a hyperparameter. Likelihood on the training data always prefers more sources. Choose the number by held-out likelihood or by a penalty. Do not choose it by the picture that looks nicest unless the picture is the decision and you say so.

One EM cycle.

Guess means and weights -> Soft-assign every point -> Update means from the soft counts -> Repeat until the likelihood stalls

Four points on a line

1. Points at 0, 0.2, 5, and 5.2. Two sources.

2. Start means at 0 and 5. The E-step assigns the first pair to the first source and the second pair to the second, almost hard, because the gap is large relative to any reasonable variance.

3. The M-step puts the means at 0.1 and 5.1. A start with both means at 0 can collapse. Run a second start before you trust a single picture.

Problem 1. The log likelihood becomes largely positive because one variance went to zero. Did the algorithm find a better cluster?

Answer. No. A Gaussian with zero variance is not a usable density. The likelihood is unbounded on this model class. Reject that run and constrain the variance.

Problem 2. You pick the number of sources by the highest training likelihood. What number will you pick?

Answer. The largest number you allow, up to one source per point, because each added source can fit leftover variation. Training likelihood is not a model-selection criterion here. Use held-out likelihood.

Lecture 10 · 180 minutes

Training a network

Backpropagation computes the gradient of the loss with respect to each weight by the chain rule. The earlier lecture did one step on a tiny net. This week is what happens when the net is deep. A gradient that is the product of many small factors vanishes, and the early layers stop moving. A product of large factors explodes, and the update destroys the weights. Initialization and the choice of nonlinearity exist to keep those products moderate.

Stochastic gradient descent uses a minibatch so the gradient is noisy and cheaper than a full pass. The step size is a hyperparameter. A step size that is too large diverges. A step size that is too small stalls inside the time you were willing to wait, which you can mistake for a finished training run. Plot the training loss. A flat line at a high value is not convergence to a good model.

Batch normalization, residual connections, and careful initialization are engineering responses to the product of factors. You should know what problem they are responding to. Using all of them because a recipe said so, without being able to say what failed before, is not a method.

The training loss going down is necessary and not sufficient. Watch the validation loss. When it rises while

training loss falls, you are overfitting. Stop, add data, or strengthen regularization. Do not keep the checkpoint with the best training loss.

What you watch while training.

1. Forward pass on a batch

2. Loss

3. Backward pass

4. Update

5. Compare validation loss

A product of three factors

1. Three layers each multiply the gradient by 0.1. The gradient at the first layer is 0.001 times the loss gradient.

2. A step size of 0.1 moves the first-layer weights by 0.0001 times the loss gradient. They are effectively frozen.

3. Replace the factors by 0.9 and the same product is about 0.73. The first layer moves. The arithmetic is the vanishing-gradient story, without a particular brand of layer.

Problem 1. Training loss falls for fifty epochs and validation loss falls for ten and then rises. Which checkpoint do you keep?

Answer. A checkpoint near epoch ten, when validation loss was lowest, not epoch fifty. Epoch fifty fits the training set better and generalizes worse. Save more than the last iterate.

Problem 2. The loss becomes NaN on step three. Name two distinct causes that produce that symptom.

Answer. A step size large enough to overflow the weights, or a loss that is undefined at the current prediction, such as a log of zero in a probability. The fix is different. Reduce the step and check the loss implementation before you add a new architecture.

Lecture 11 · 180 minutes

Scores and calibration

Accuracy hides the base rate. The evaluation lecture computed precision and recall on a rare positive class. This week adds the score itself. Many models emit a number that is not a probability. A threshold turns the number into a decision. The right threshold depends on the costs of the two mistakes, which are part of the problem, not a default of 0.5.

A proper scoring rule rewards calibrated probabilities. Log loss does. Accuracy does not. You can improve accuracy by predicting the majority class and be useless on the rare class. Report the score that matches the decision. If someone will threshold at a cost ratio, show precision and recall at that threshold, not at the threshold that makes the paper look balanced.

Calibration plots bin the predicted probabilities and compare them to the observed frequency. A model that says 0.9 and is right 60 percent of the time is overconfident. Recalibration on a validation set can repair the numbers without retraining the features. It cannot repair a model that ranks the cases backwards.

A single test set gives a noisy estimate. If you need a comparison, use a second split or a bootstrap and show the range. A 0.01 difference on a hundred examples is not a result. Compute the counts before you write 'better.'

Two reports from one model.

A decision

Pick a threshold from costs

Report precision and recall there

Accuracy alone is not enough

A probability

Log loss on held-out data

A calibration curve

Do not threshold first

One hundred people, ten positive

1. The model flags 20 people. Of the 10 positives, it finds 8. Precision is 8/20 = 0.4. Recall is 8/10 = 0.8.

2. Accuracy is (8 true positives + 78 true negatives) / 100 = 0.86. Predicting nobody yields accuracy 0.90 and recall 0. A higher accuracy can be a worse detector.

3. If a false positive costs 1 and a missed positive costs 9, the 8 true positives and 2 misses and 12 false positives have a cost you can compute: 2 * 9 + 12 * 1 = 30. Compare that cost to another threshold instead of comparing accuracies.

Problem 1. You sweep thresholds on the test set and report the best F1. What did you use the test set for?

Answer. For model selection. The F1 you report is optimistic. Choose the threshold on validation data and touch the test set once.

Problem 2. The model outputs scores between 0 and 1 that are badly calibrated, and the ranking of patients is correct. What can you fix without new features?

Answer. The mapping from score to probability, fit on validation data. You cannot fix a reversed ranking by recalibration. Check the ranking first. If it is right, a monotone map can repair the probabilities.

Lecture 12 · 180 minutes

Leakage

Leakage is information from the test label, or from the future, entering the training procedure. It produces optimistic numbers that collapse on a real deployment. The common cases are concrete. Scaling features using the mean of the whole dataset before the split. Selecting features by correlation with the label on all rows. Duplicating a person across train and test. Using a timestamp from after the outcome.

The split unit is the unit that will be new later. If you will see new patients, split by patient, not by visit. Two visits from the same patient, one in train and one in test, let the model recognize the patient. The test score then measures memorization of people, which you will not have when a new person arrives.

Any preprocessing that looks at labels or at other rows is part of the model and must be fit on the training fold only. A pipeline object is useful because it attaches the scaler to the fold. A notebook that scales, then splits, is the bug.

When a result looks too good for the features you have, assume leakage and try to find it before you assume a breakthrough. A model that predicts tomorrow's value from a column that already contains tomorrow's value is not a model.

A split that does not leak.

1. Choose the split unit

2. Hold out the test units

3. Fit every preprocessor on train only

4. Evaluate once

A column that is the label

1. The table has a column total that equals the sum of the feature you meant to predict and a tax that is a fixed fraction of it.

2. A model given total predicts the target with almost no error. The column is the label wearing a hat.

3. Drop it. If the error stays tiny, look for another copy of the label: an id that orders the classes, a timestamp of the decision, a duplicate row.

Problem 1. You normalize each feature by the maximum in the column, computed before the split. How can that use the test set?

Answer. The maximum may be a test row. The training features were divided by a number that depends on the test set. Fit the maximum on the training rows only, and apply that same number to the test rows.

Problem 2. Accuracy is 0.99 and the positive class is 50 percent of the data. You did not expect the features to be that informative. What do you check first?

Answer. Leakage: duplicate rows, a column derived from the label, and whether the split unit matches the deployment unit. Only after those checks fail do you treat 0.99 as a result.

Lecture 13 · 180 minutes

Prediction is not a cause

A predictive model estimates an outcome from features as they co-occur in the data. A causal claim says what would happen if you changed a feature. Those are different questions. A model that predicts readmission from the number of medications is not a reason to take the medications away. The medications may mark the severity that causes readmission.

If you need a causal answer, you need a design: a randomized assignment, a natural experiment, or an explicit assumption about what is unobserved. The algorithm does not supply the assumption. Writing 'controlled for' and listing covariates is not a design unless the list is justified.

Distribution shift is the predictive version of the same humility. A model trained in one hospital sees different measurements in another. Accuracy on a random split of the first hospital does not transfer. Collect a sample from the place you will deploy, or restrict the claim to the place you measured.

The write-up should contain the sentence: this predicts, it does not say what to change. If you do intend an intervention, the sentence should be replaced by the assumption that makes the intervention valid, and by the measurement you would accept as a refutation.

Two questions, two methods.

Prediction

Features as observed

A held-out sample from the same process

Accuracy, calibration, cost

Intervention

A change you impose

An assumption or a randomization

An outcome measured after the change

Medications and readmission

1. Patients with more medications are readmitted more often. A model uses the count and ranks patients well.

2. A policy that reduces medications in order to reduce readmission is a different claim. The count may be a stand-in for illness.

3. The predictive model can still be the right tool for scheduling follow-up. It is the wrong tool for deciding which drug to cancel, until someone states an assumption you can test.

Problem 1. You add twenty covariates and the coefficient on medications shrinks. May you call the remaining coefficient a causal effect?

Answer. No. Shrinking is consistent with confounding and also with having removed part of the pathway you meant to measure. A coefficient from a regression is a causal effect only under assumptions you have to state. The shrinkage itself is not those assumptions.

Problem 2. The model was trained on morning labs and will be run on afternoon labs from a new clinic.

What number do you not yet have?

Answer. The error on afternoon labs from that clinic. The morning test set does not provide it. Measure a sample, or do not claim the morning accuracy.

Lecture 14 · 180 minutes

A study

The last week is a study small enough to finish and strict enough to defend. Pick a dataset you can describe in one paragraph: what a row is, how it was collected, and what a prediction is for. Split by the unit that will be new later. Fit a linear model and one nonlinear model. Choose hyperparameters only on the training side.

Report the base rate, the cost-weighted error or precision and recall at a stated threshold, and the same numbers for a baseline that predicts the majority class or the mean. A model that does not beat that baseline on the metric you care about has not earned a diagram.

Include one failure. A slice of the data where the model is worse, or a feature you removed because it leaked. A study with no failure mode has not been inspected.

The hours for this course include this study. The opening lectures are the vocabulary. This write-up is the evidence that you can use it without fooling yourself.

The study, in order.

Describe the row and the decision

Split by the deployment unit

Baseline, then two models

One failure you can point at

A baseline that is easy to forget

1. Ninety of 100 labels are negative. The majority predictor scores accuracy 0.90 and recall 0.

2. Your model scores accuracy 0.91 and recall 0.2. On accuracy it barely wins. On recall it is a different tool.

3. If the decision needs recall, report 0.2 against 0, not 0.91 against 0.90. The baseline is part of the result.

Problem 1. You try five model families on the test set and keep the best accuracy. How should you label that accuracy?

Answer. As a tuned result, not as a confirmatory test. The test set selected the family. Either nest the selection inside the training side and re-evaluate once, or say the number is exploratory.

Problem 2. The nonlinear model matches the linear model on every metric you chose. What do you conclude?

Answer. On this split and these metrics, the extra capacity did not pay. Prefer the linear model and say so. Do not keep the nonlinear model because it is more modern. Equality is a result.